第 20 章：Prompt 工程与模板设计

版本: v2.0（润色版）
作者: 内容撰写专家（LLM 原理 + Prompt 工程方向）
状态: 润色完成
最后更新: 2026-03-23

【本章导读】

Prompt 是 Agent 与 LLM 沟通的语言，设计质量直接影响输出效果。本章讲解 Prompt 结构设计、Few-shot 与 Chain-of-Thought 原理、模板迭代方法和系统提示词设计。掌握这些技能后，你能够设计可复用、可迭代、可测试的 Prompt 模板，显著提升 Agent 的输出质量和稳定性。

20.1 Prompt 结构设计

三种角色提示词

现代 LLM API 支持三种角色的提示词，理解它们的区别和用途是设计高质量 Prompt 的基础。

系统提示词（System Prompt）： - **定义**：定义 Agent 的角色、能力边界、行为准则 - **作用时机**：对整个对话生效，优先级最高 - **内容示例**：「你是专业漫剧策划助手，擅长帮助作者梳理创意。你必须基于作者确认的设定进行创作，不能自行添加未确认的设定。」 - **最佳实践**：放在对话开头，500-1000 Token，简洁清晰

用户提示词（User Prompt）： - **定义**：当前请求或问题 - **作用时机**：每轮对话变化，携带具体任务 - **内容示例**：「我想创作一部玄幻漫剧，主角是一个失去记忆的少年，背景是一个灵力分级的世界。」 - **最佳实践**：结构化表达，关键信息突出

助手提示词（Assistant Prompt）： - **定义**：历史回复，用于多轮对话上下文 - **作用时机**：构建对话历史，让 LLM 理解上下文 - **内容示例**：「好的，我来帮你梳理这部玄幻漫剧的设定。首先，关于灵力分级体系，你设想的是几级制？」 - **最佳实践**：保留最近 10-20 轮，超出部分摘要或移除

为什么区分三种角色： - **优先级不同**：系统提示词 > 用户提示词 > 助手提示词 - **作用范围不同**：系统提示词影响整体行为，用户/助手提示词是对话内容 - **管理方式不同**：系统提示词固定，用户提示词动态，助手提示词滑动窗口

常见误区：把所有内容都放在用户提示词。实际系统提示词更适合定义角色和边界，用户提示词专注于当前任务。分离后，系统提示词只需设置一次，用户提示词可灵活变化。

漫剧创意沟通案例： - 系统提示词：定义「专业漫剧策划助手」角色，设定行为准则 - 用户提示词：作者的创意描述（每轮变化） - 助手提示词：Agent 的历史回复（保留最近 10 轮）

上下文组织方法

上下文窗口有限，如何组织信息让 LLM 最有效利用是关键挑战。

LLM 注意力分布特点： - **首尾效应**：LLM 对开头和结尾的信息注意力更高，中间部分容易被忽略 - **近因效应**：最近的信息（对话末尾）影响力更大 - **结构效应**：结构化信息（标题、列表）比大段文字更容易理解

组织策略：

策略 1：重要信息放两端 - 核心设定放在系统提示词开头 - 关键约束放在用户提示词末尾 - 中间放置参考信息或背景说明

策略 2：结构化组织 - 用标题分隔不同区块（如「## 世界观设定」「## 角色列表」） - 用列表呈现并列信息（如角色属性） - 用分隔符（如 `---` ）区分不同部分

策略 3：相关性排序 - 最相关信息放前面 - 次要信息放后面 - 无关信息直接移除

实践参数建议： - 系统提示词：500-1000 Token（定义角色和核心规则） - 用户提示词：根据剩余窗口动态调整（通常 1000-3000 Token） - 对话历史：保留最近 10-20 轮（约 2000-4000 Token）

漫剧大纲生成案例： `[开头] 系统提示词：核心设定（世界观规则、主角基本信息） [中间] 参考示例：2 个高质量大纲示例 [末尾] 当前任务：「请根据以上设定，生成第 1 章细纲，要求覆盖以下剧情点：...」`

为什么这样组织：核心设定在开头确保 LLM 理解基础，示例在中间提供参考模式，任务描述在末尾确保 LLM 清楚当前目标。

常见误区：认为上下文顺序不重要。实际 LLM 对不同位置的信息关注度不同，关键信息应放在注意力高的位置。

关键信息放置策略

长上下文中关键信息容易被忽略，需要特殊处理确保 LLM 正确引用。

问题：LLM 可能「忘记」或忽略关键信息，导致输出不符合要求。

策略 1：重复强调 - 在系统提示词中定义关键规则 - 在用户提示词中再次提醒 - 多轮对话中定期重申

示例：`系统提示词：「你必须基于作者确认的设定进行创作。」 用户提示词：「请根据以上设定（特别是灵力分级规则）生成第 3 章。」`

策略 2：显式标注 - 用标记突出关键信息（如「【重要】」「⚠️」） - 用大写或加粗强调（如「**必须**遵守」） - 单独列出关键约束

示例：`【重要约束】 1. 不能添加未确认的角色 2. 灵力等级不能超过主角当前等级 3. 本章必须包含「主角发现秘密」剧情点`

策略 3：单独提取 - 将关键信息放在专门的区块（如「## 核心设定」） - 用结构化格式呈现（如表格、列表） - 与背景信息分离

示例：```

核心设定（必须遵守）

| 设定项 | 内容 | |--------|------| | 灵力等级 | 1-9 级，主角当前 3 级 | | 世界观 | 玄幻，不能出现科技元素 | | 主角 | 失忆少年，名字未定 | ```

验证方法： - 测试 LLM 是否能正确引用关键信息 - 检查输出是否违反核心约束 - 发现问题后调整放置位置或强调方式

漫剧世界观规则案例：「灵力等级体系」在系统提示词定义，在每次生成时重复提醒。发现 LLM 仍有时违反时，在用户提示词末尾再次强调（「⚠️ 注意：主角当前 3 级，不能战胜 5 级以上敌人」）。

常见误区：认为「说过一次 LLM 就会记住」。实际需要在关键位置重复强调，尤其是长对话场景。

20.2 Few-shot 与 Chain-of-Thought

Few-shot Learning 原理

Few-shot Learning（少样本学习）通过提供少量示例，让 LLM 学习任务模式，是提升输出质量最有效的方法之一。

定义：提供 1-5 个输入 - 输出示例，让 LLM 理解任务要求和输出格式。

为什么有效： - **直观展示**：示例比文字描述更直观，LLM 能直接学习输入 - 输出映射 - **模式识别**：LLM 擅长识别模式，示例展示了期望的输出模式 - **减少歧义**：示例消除了文字描述可能存在的歧义

示例选择原则： - **代表性**：选择能代表任务典型情况的示例 - **覆盖边界**：包含边界情况（如最短输出、最复杂输出） - **质量优先**：示例质量必须高，LLM 会模仿示例的质量 - **多样性**：多个示例应展示不同情况（如不同题材、不同长度）

示例顺序： - **从简单到复杂**：帮助 LLM 逐步理解任务 - **从通用到特殊**：先展示通用模式，再展示特殊情况 - **最后一个示例最重要**：LLM 对最后一个示例印象最深，应放最具代表性的

漫剧设定生成案例： ``` 示例 1（简单，玄幻题材）： 输入：「我想创作一部玄幻漫剧，主角是一个失去记忆的少年。」 输出：{世界观：灵力分级世界，1-9 级... 主角：失忆少年，当前 3 级...}

示例 2（复杂，都市题材）： 输入：「我想创作一部都市超能力漫剧，主角能控制时间。」 输出：{世界观：现代都市，隐藏超能力者... 主角：时间控制者，能力限制...}

当前任务： 输入：「我想创作一部奇幻漫剧，主角是一个被诅咒的公主。」 输出： ```

常见误区：认为示例越多越好。实际超过 5 个收益递减，且消耗大量 Token。2-3 个高质量示例通常足够。

参数建议： - 示例数量：2-3 个（平衡质量和成本） - 示例长度：每个 200-500 Token（足够展示模式，不浪费） - 示例位置：用户提示词中，当前任务之前

Chain-of-Thought（思维链）

Chain-of-Thought（CoT，思维链）让 LLM 展示推理过程，而不是直接给答案，能显著提升复杂任务的准确性。

定义：要求 LLM 逐步展示思考过程，最后给出答案。

为什么有效： - **强制分解**：CoT 强制 LLM 将复杂问题分解为多个步骤 - **减少跳跃**：避免 LLM 跳跃式推理，减少错误 - **可检查性**：推理过程可见，能发现哪一步出错 - **自我纠正**：LLM 在展示过程中可能自我发现错误

触发方式： - **Few-shot 示例**：示例中包含推理步骤（最有效） - **指令明确要求**：「请逐步思考」「让我们一步步分析」 - **结构化提示**：「步骤 1... 步骤 2... 步骤 3...」

适用场景： - **数学推理**：计算、逻辑题 - **复杂规划**：多步骤任务分解 - **逻辑推理**：因果关系分析 - **质量检查**：一致性校验

不适用场景： - **简单问答**：增加 Token 消耗，无必要 - **格式化输出**：直接输出结果更高效 - **创意生成**：过度推理可能限制创意

漫剧大纲规划案例： ``` 请按以下步骤思考：

步骤 1：分析创意核心 - 题材：玄幻 - 主题：成长与复仇 - 核心冲突：主角 vs 杀父仇人

步骤 2：确定三幕结构 - 第一幕（1-5 章）：主角觉醒，发现身世 - 第二幕（6-15 章）：修炼成长，收集盟友 - 第三幕（16-20 章）：最终决战，复仇成功

步骤 3：分配章节内容 - 第 1 章：主角出场，遭遇危机 - 第 2 章：觉醒能力，遇到导师 - …

最终输出：[章节大纲 JSON] ```

常见误区：认为 CoT 适合所有任务。实际简单任务用 CoT 增加 Token 消耗，无必要。应根据任务复杂度选择。

参数建议： - 复杂任务：启用 CoT，要求展示推理过程 - 简单任务：直接输出结果 - 中间场景：可选 CoT，或要求简化的推理摘要

引导逐步推理

设计有效的 CoT 提示需要技巧，以下是实践方法。

指令设计： - **明确步骤**：「请按以下步骤思考：1… 2… 3…」 - **定义每步内容**：说明每步应分析什么 - **设置检查点**：在关键步骤要求确认

示例： ```请按以下步骤分析剧情逻辑：

1. 列出本章所有事件（按时间顺序）
2. 分析事件之间的因果关系
3. 检查是否有矛盾（如角色位置冲突、时间冲突）
4. 指出需要修改的地方

请逐步输出，最后给出修改建议。```

中间检查： - **暂停确认**：「请确认以上分析是否正确，我再继续」 - **用户干预**：允许用户在中间步骤提出修改 - **分支处理**：如果某步发现问题，先解决再继续

输出格式： - **分离推理和答案**：用分隔符区分（如 `---`） - **结构化推理**：用标题、列表呈现推理过程 - **便于解析**：最终答案用固定格式（如 JSON）

示例： ```

推理过程

步骤 1：事件列表

1. 主角遇到敌人
2. 发生战斗
3. 主角获胜

步骤 2：因果关系

- 敌人出现是因为...
- 战斗爆发是因为...

步骤 3：矛盾检查

- 无矛盾

最终答案

{ "status": "pass", "suggestions": [] } ```

调试方法： - 检查推理过程中哪一步出错 - 针对性改进该步的指令 - 重新测试验证

漫剧剧情逻辑检查案例：要求 LLM 先列出所有事件、再分析因果关系、最后指出矛盾点。发现 LLM 有时跳过因果关系直接给结论，改进指令为「必须完成每一步才能进入下一步」，问题解决。

常见误区：要求 CoT 但不给示例。实际 LLM 可能不理解你想要的推理深度。最好提供 1-2 个包含推理过程的示例。

20.3 模板设计方法与迭代

模板结构设计

Prompt 模板应模块化设计，便于复用、调试和迭代。

模块化设计原则： - **单一职责**：每个模块负责一个功能 - **可插拔**：模块可独立启用/禁用 - **可组合**：多个模块可组合使用

推荐模块：

| 模块名 | 功能 | 是否必填 | 典型 Token 数 | |-------|------|---------|-------------| | 系统指令 | 定义角色和边界 | 必填 | 300-500 | | 设定区块 | 世界观、角色等设定 | 可选 | 500-1500 | | 示例区块 | Few-shot 示例 | 可选 | 400-1000 | | 任务描述 | 当前任务说明 | 必填 | 200-500 | | 输出格式 | 期望输出格式 | 必填 | 100-300 | | 约束条件 | 特殊约束 | 可选 | 100-300 |

变量占位符： - **语法**：用 `{variable}` 或 `{{variable}}` 标记动态内容 - **命名规范**：有意义的名称（如 `{user_creative_idea}` 而非 `{var1}`） - **类型标注**：可选，帮助理解（如 `{chapter_number: int}`）

示例模板： ```

系统指令

你是专业漫剧策划助手，基于作者确认的设定进行创作。

设定区块

{world_setting} {character_list}

示例区块

{few_shot_examples}

任务描述

请根据以上设定，生成第 {chapter_number} 章细纲。 创意描述：{user_creative_idea}

输出格式

请输出 JSON 格式，包含以下字段： - chapter_title: 章节标题 - plot_points: 剧情点列表 - word_count: 预计字数

约束条件

{constraints} ```

条件逻辑： - **可选模块**：根据场景选择是否启用 - **动态内容**：根据变量值调整内容

示例： `如果有设定区块： 加入「请严格遵循以上设定」 否则： 加入「请根据创意描述自由发挥」`

长度控制： - 每个模块设置最大 Token 数 - 渲染前检查总长度，超出时压缩或移除可选模块 - 优先级：系统指令 > 任务描述 > 输出格式 > 设定区块 > 示例区块

常见误区：模板过于复杂。实际应保持简洁，便于调试和迭代。初期 3-5 个模块足够，后续根据需要扩展。

变量与占位符

变量管理是模板设计的核心，良好的变量设计能显著提升可维护性。

变量类型：

| 类型 | 定义 | 示例 | 管理方式 | |------|------|------|---------| | 静态变量 | 固定内容，不随任务变化 | 系统指令、输出格式 | 模板中硬编码 | | 动态变量 | 每任务变化 | 用户创意、章节号 | 运行时填充 | | 计算变量 | 根据其他变量生成 | 设定摘要、上下文摘要 | 代码计算后填充 |

命名规范： - **有意义**： `{user_creative_idea}` 而非 `{var1}` - **一致性**：同类变量用相同前缀（如 `setting_*` 、 `character_*` ） - **可读性**：下划线分隔（ `chapter_number` ），不用驼峰（ `chapterNumber` ）

默认值： - **可选变量**：设置默认值，避免渲染失败 - **安全默认**：选择最保守的默认值

示例： `{constraints: "无特殊约束"} # 默认无约束 {few_shot_examples: ""} # 默认无示例`

验证机制： - **必填检查**：渲染前检查必要变量是否存在 - **类型检查**：验证变量类型（字符串、数字、列表） - **范围检查**：验证变量值在合理范围（如章节号 1-100）

漫剧章节生成模板变量： `{chapter_number}: 章节号（动态变量，必填） {plot_points}: 本章剧情点列表（动态变量，必填） {character_list}: 出场角色列表（动态变量，必填） {world_setting}: 世界观设定（静态变量，可选） {constraints}: 特殊约束（动态变量，默认"无"）`

常见误区：变量名不清晰。实际会影响后续维护和调试。花时间是命名是值得的投资。

版本管理与迭代流程

Prompt 模板需要持续迭代，版本管理确保可追溯、可回滚。

版本命名： - **语义化版本**： `v1.0` 、 `v1.1` 、 `v2.0` - **规则**： - 主版本号（v1→v2）：重大变更，可能影响输出格式 - 次版本号（v1.0→v1.1）：小幅改进，输出格式兼容 - 修订号（v1.0.1）：bug 修复，无明显变化

变更记录： 每次修改记录以下内容： - 变更内容 - 变更原因 - 效果对比（前后指标） - 变更日期

示例： ```markdown

v2.0 (2026-03-20)

- 变更：添加 Few-shot 示例（2 个）
- 原因：通过率从 85% 降至 70%，需要示例引导
- 效果：通过率提升至 95%
- 变更人：Agent-内容撰写

v1.1 (2026-03-15)

- 变更：优化输出格式描述
- 原因：LLM 有时输出额外字段
- 效果：格式错误率从 15% 降至 5% ```

迭代流程：`1. 发现问题（通过率低、质量差） ↓ 2. 分析原因（检查错误类型、分布） ↓ 3. 设计改进（修改 Prompt、添加示例） ↓ 4. 小范围测试（抽样 20-50 次） ↓ 5. 评估效果（通过率、质量评分） ↓ 6. 决定是否发布 ↓ ├─ 效果提升 → 发布新版本 └─ 效果下降 → 回滚，重新分析`

回滚机制： - 保留历史版本文件 - 新版本效果差时快速回滚 - 记录回滚原因

文件组织：`prompts/ ├── chapter-outline-v1.0.md ├── chapter-outline-v1.1.md ├── chapter-outline-v2.0.md # 当前版本 └── CHANGELOG.md # 变更记录`

漫剧设定生成 Prompt 迭代案例： - v1.0：通过率 85%，主要问题是格式错误 - v1.1：优化输出格式描述，通过率 90% - v2.0：添加 Few-shot 示例，通过率 95% - 总迭代次数：3 次，耗时 1 周

常见误区：不记录变更历史。实际效果变差时无法回滚，也不知道什么改进有效。

A/B 测试判断优劣

A/B 测试是评估 Prompt 改进效果的科学方法，避免主观判断。

测试设计： - **同一批输入**：确保变量一致 - **两个版本**：A（旧版本）和 B（新版本） - **独立评估**：分别运行，比较输出

评估指标：

| 指标 | 定义 | 测量方式 | 目标值 | |------|------|---------|------| | 通过率 | 符合格式要求的比例 | 自动解析检查 | >90% | | 质量评分 | 输出质量（人工或 LLM 评估） | 1-5 分 | >4.0/5.0 | | Token 用量 | 平均消耗 Token | API 返回统计 | <3000 Token/次 | | 响应时间 | 平均生成时间 | 计时统计 | P50<3 秒，P90<5 秒 | | 人工修改率 | 需要人工修改的输出比

例 | 修改次数/总输出次数 | <15% | | A/B 测试显著性 | 统计检验 p 值 | 卡方检验或 t 检验 | p<0.05（95% 置信度） |

样本量： - **最小样本**：至少 100 次，避免偶然性 - **统计显著性**：用卡方检验或 t 检验判断差异是否显著 - **置信度**：通常 95%（p < 0.05）

测试流程： `1. 准备测试集（100 条代表性输入） ↓ 2. 运行 A 版本，记录结果 ↓ 3. 运行 B 版本，记录结果 ↓ 4. 计算指标（通过率、质量评分等） ↓ 5. 统计检验（判断差异是否显著） ↓ 6. 决策 ↓ ├─ B 显著更好 → 采用 B ├─ 无显著差异 → 保留 A（避免不必要变更） └─ B 更差 → 放弃 B，重新设计`

漫剧大纲生成 A/B 测试案例： - 测试集：100 个创意描述 - A 版本（无示例）：通过率 92%，质量评分 3.8 - B 版本（添加 2 个示例）：通过率 95%，质量评分 4.2 - 统计检验：p < 0.05，差异显著 - 决策：采用 B 版本

常见误区：只测几次就下结论。实际需要足够样本量，避免偶然性影响判断。

20.4 系统提示词设计

人格设定

系统提示词定义 Agent 的人格，影响整体交互风格。

角色定义： - **专业领域**：「你是 XX 领域的专家」 - **职责描述**：「你的职责是 XX」 - **能力范围**：「你擅长 XX，不擅长 XX」

示例： `你是专业漫剧策划助手，拥有 10 年漫剧创作经验。 你的职责是帮助作者梳理创意、编写设定、规划剧情。 你擅长玄幻、都市、奇幻题材，对历史题材了解有限。`

语气风格： - **正式**：「根据您的要求...」 - **友好**：「好的，我来帮你...」 - **幽默**：「这个创意太棒了，让我们...」 - **简洁**：直接给出结果，无多余描述

选择原则：与目标用户群体匹配。专业开发者可能偏好简洁，普通用户可能偏好友好。

知识边界： - **明确限制**：「你只知道 XX 领域的知识」 - **不确定处理**：「不确定的内容要说不知道」 - **拒绝越界**：「超出能力范围的任务要礼貌拒绝」

示例： `你只知道漫剧创作相关知识。 如果用户询问其他领域（如医学、法律），请礼貌告知无法回答。 不确定的内容要说「我不确定」，不要编造。`

一致性维护： - 多轮对话中保持人格一致 - 避免前后矛盾（如前一轮说「我不确定」，后一轮又给出确定答案） - 定期审查对话历史，发现不一致及时修正

漫剧策划助手人格设定案例：`角色：专业漫剧策划助手 语气：友好、耐心、有创意 知识边界：漫剧创作相关（设定、大纲、剧情） 限制：不能生成色情暴力内容 不确定处理：「这个设定我需要更多信息才能确定」`

常见误区：人格设定过于模糊。实际需要具体描述（如「用简洁的中文回复」「每次回复不超过 200 字」）。

边界与约束

系统提示词应明确定义能力边界和约束，防止 LLM 越界。

能力边界：- **能做什么**：「你只能 XX」「你的职责是 XX」 - **不能做什么**：「你不能 XX」「禁止 XX」

示例：`你只能基于作者确认的设定进行创作。 不能自行添加未确认的角色、设定、剧情。 如果用户要求添加新设定，必须先询问确认。`

安全约束：- **内容安全**：「不能生成违法、色情、暴力内容」 - **隐私保护**：「不能泄露用户隐私信息」 - **合规要求**：「必须遵守 XX 法规」

示例：`不能生成以下内容： - 违法、色情、暴力内容 - 歧视、仇恨言论 - 用户隐私信息（如真实姓名、地址） 如发现用户请求涉及以上内容，请礼貌拒绝。`

格式约束：- **输出格式**：「必须用 JSON 格式」「必须分点回答」 - **长度限制**：「回复不超过 200 字」「列表不超过 10 项」 - **语言要求**：「用中文回复」「专业术语保留英文」

示例：
`输出必须为 JSON 格式，包含以下字段： - status: "pass" 或 "fail" - suggestions: 修改建议列表（最多 5 条） 回复必须用中文，专业术语保留英文原名。`

越界处理：- **检测越界**：识别用户请求超出边界 - **礼貌拒绝**：「抱歉，这个任务超出我的能力范围」 - **提供替代**：「但我可以帮你 XX」

示例：`用户：「帮我写一部漫剧的完整正文」 回复：「抱歉，我的职责是帮助规划剧情和设定，不直接撰写正文。 但我可以帮你生成详细细纲，你可以根据细纲自行创作。」`

漫剧生成系统约束案例：```能力边界：- 能：设定管理、大纲规划、细纲分解 - 不能：直接撰写正文、生成图片

安全约束：- 不能生成色情暴力内容 - 不能泄露用户隐私

格式约束：- 设定输出必须为 JSON 格式 - 大纲输出必须包含章节标题和剧情点```

常见误区：只定义能做什么，不定义不能做什么。实际边界同样重要，防止 LLM 过度发挥。

输出格式控制

系统提示词应明确定义输出格式，便于后续解析和处理。

格式定义： - **格式类型**：JSON、Markdown、纯文本、XML - **结构说明**：每个字段的含义、类型、是否必填 - **示例展示**：提供一个符合格式的输出示例

JSON 格式示例： ``` 请输出 JSON 格式，包含以下字段：

{ "chapter_title": "章节标题（字符串）", "plot_points": ["剧情点 1", "剧情点 2", ...], "word_count": 预计字数（整数）, "characters": ["出场角色 1", "出场角色 2", ...] }

注意： - 只输出 JSON，不要有其他内容 - 确保 JSON 格式正确，能被解析 ```

字段说明： - **字段名**：清晰有意义（`chapter_title` 而非 `title`） - **类型**：字符串、整数、列表、布尔值 - **必填/可选**：明确标注 - **约束**：长度限制、取值范围

示例展示： `示例输出： { "chapter_title": "觉醒之夜", "plot_points": [ "主角遭遇敌人袭击", "危急时刻能力觉醒", "神秘人出现并救下主角" ], "word_count": 3000, "characters": ["主角", "敌人", "神秘人"] }`

为什么示例重要：示例比文字描述更直观，LLM 能直接模仿示例格式。

容错处理： - **格式错误**：LLM 可能输出格式错误（如 JSON 缺少引号） - **解析策略**： - 尝试修复（如自动补全引号） - 重试（要求 LLM 重新输出） - 兜底（用正则提取关键信息）

漫剧设定输出格式案例： `请输出 JSON 格式： { "name": "角色名（字符串）", "description": "角色描述（字符串，100-300 字）", "attributes": { "age": 年龄（整数）, "gender": "性别（男/女/其他）", "ability": "能力描述（字符串）" }, "background": "背景故事（字符串，可选）" }`

常见误区：只描述格式不给示例。实际示例比文字描述更有效，应始终提供至少 1 个示例。

20.5 简单举例

案例设计

- **案例名称**：漫剧创意沟通的 Prompt 模板
- **涉及知识点**：Prompt 结构设计、Few-shot 与 Chain-of-Thought、模板设计方法与迭代、系统提示词设计
- **案例目标**：帮助理解如何设计一个高效的 Prompt 模板来梳理漫剧创意
- **案例内容要点**：

- 场景描述：设计一个 Prompt 模板，帮助作者梳理漫剧创意，输出结构化的设定文档
- 技术应用：定义「专业漫剧策划助手」系统提示词，提供 2 个高质量 Few-shot 示例，设计模板变量，经过 3 轮 A/B 测试迭代优化
- 效果说明：创意收集效率提升，输出结构化程度从 60% 提升至 90%，作者满意度从 3.5 分提升至 4.3 分（5 分制）
- **注意事项**：不展开 A/B 测试的具体设计细节（见第 16 章）

知识来源: - Chain-of-Thought 论文：Chain-of-Thought Prompting Elicits Reasoning (Wei et al., 2022) - https://arxiv.org/abs/2201.11903 - LangChain Prompt 官方文档 - https://python.langchain.com/docs/modules/model_io/prompts/prompt_templates - OpenAI Prompt Engineering 最佳实践 - https://platform.openai.com/docs/guides/prompt-engineering

---

修改记录: - v2.0 (2026-03-23): 润色版 — 句子简化、删除重复、优化结构、统一语气 - v1.1 (2026-03-22): 根据编辑统筹意见修改 — 规范知识来源格式 - v1.0 (2026-03-22): 初稿完成